

Event Horizon First PR Report

Generated: 2026-05-30T00:42:00.745Z

Executive Summary

Event Horizon now has a first playable vertical slice on `feat/first-playable`: a mobile-first Vite + PixiJS v8 canvas game where the player taps, swipes, and captures dark-energy orbs to delay a black-hole collapse. The run is deterministic from seed plus input timings, uses a fixed 60 Hz simulation step, records replay payloads, exports a three-frame share poster, and includes a minimal score-submit endpoint. The repo builds locally, unit tests pass, mobile Chrome smoke tests pass, and screenshots were captured from the actual running build.

Assumptions

- The repo was greenfield except for the initial commit, so a plain static Vite scaffold was chosen.
- The target playable slice is a prototype, not a balanced production game.
- `2026-05-29` is used as the alpha seed date because it was provided in the task context.
- GitHub Pages is the public static target at `/event-horizon/`; Netlify is supported with a root base path.
- A custom deterministic simulation is sufficient for radial gravity, tether capture, flybys, and shadow arms; no Matter.js or Planck.js dependency is justified yet.
- The Netlify endpoint validates and acknowledges replay payloads only; storage and anti-cheat are backlog items.

Chosen Stack and Why

- Vite + TypeScript: smallest viable static scaffold, fast local dev, and straightforward GitHub Pages base-path support.
- PixiJS v8: direct WebGL-preferred 2D rendering with a scene graph and generated textures. The implementation follows the PixiJS v8 async `Application.init()` pattern and `preference: 'webgl'` option from the official PixiJS docs.
- Custom simulation: deterministic, low-allocation motion was simpler than integrating a physics engine for this slice.
- Vitest: fast deterministic simulation and endpoint unit tests.
- Playwright with Chrome: local mobile/touch smoke checks for the target browser.
- Netlify Functions: minimal serverless score endpoint in-repo, matching Netlify's default `/.netlify/functions/<name>` routing.
- Google Apps Script sample: alternate deploy path using `doPost` and `ContentService` JSON output.

Sources used: [PixiJS Application docs](https://pixijs.com/8.x/guides/components/application), [PixiJS official skills repo](https://github.com/pixijs/pixijs-skills), [Netlify Functions docs](https://docs.netlify.com/functions/get-started/), [Google Apps Script Content Service docs](https://developers.google.com/apps-script/guides/content), and [Vite base config docs](https://main.vitejs.dev/config/shared-options).

Implementation Plan and Estimated Hours

1. Repo inspection and AGENTS.md: 0.25h
2. Vite + TypeScript + PixiJS scaffold: 0.75h
3. Deterministic simulation, fixed-step loop, RNG, and replay payloads: 2.0h
4. PixiJS scene, galaxy background, black hole, orbs, tethers, flybys, shadow arms, HUD: 2.0h
5. Tap/swipe input handler and logical viewport scaling: 1.0h
6. Collapse animation and posterizer export: 1.25h
7. Netlify score function and GAS sample: 0.75h
8. Unit, e2e, score endpoint, and artifact capture tests: 1.25h
9. README, PR body, PDF report, and final polish: 1.25h

Total estimate: 10.5h

Prioritized Backlog

1. Add durable score storage and abuse limits for Netlify or GAS.
2. Add replay playback UI and a deterministic replay verifier in CI.
3. Balance phase thresholds, orb spawn curves, and capture scoring through playtest data.

4. Add visual capture streak feedback and better end-run poster composition.
5. Add sound, haptics, and reduced-motion options.
6. Add GitHub Actions for build, lint, unit tests, and Playwright smoke tests.
7. Add accessibility affordances for non-touch desktop play.

Git and PR Commands

```
git switch -c feat/first-playable
npm install
npm run build
npm run lint
npm run test
npm run test:e2e
npm run score:test
npm run capture:artifacts
npm run report:pdf
git add .
git commit -m "Build first playable Event Horizon slice"
git push -u origin feat/first-playable
gh pr create --base main --head feat/first-playable --title "Build first playable Event Horizon slice" --body-file docs/pr-body.md
```

File Tree

```
AGENTS.md
README.md
docs/artifacts/collapse-mobile.jpg
docs/artifacts/gameplay-mobile.jpg
docs/artifacts/share-poster.jpg
docs/first-pr-report.md
docs/pr-body.md
eslint.config.js
gas/score-submit.gs
index.html
netlify.toml
netlify/functions/score-submit.mjs
package-lock.json
package.json
playwright.config.ts
scripts/capture-artifacts.mjs
scripts/generate-report-pdf.mjs
scripts/test-score-submit.mjs
src/game/EventHorizonGame.ts
src/game/FixedStepLoop.ts
src/game/InputHandler.ts
src/game/Simulation.ts
src/game/constants.ts
src/game/math.ts
src/game/posterizer.ts
src/game/rng.ts
src/game/scoreClient.ts
src/game/types.ts
src/main.ts
src/styles.css
src/vite-env.d.ts
tests/e2e/playable.spec.ts
tests/mjs.d.ts
tests/score-submit.test.ts
tests/simulation.test.ts
tsconfig.json
vite.config.ts
```

Changed and Created File Notes

- `AGENTS.md`: practical repo instructions for future Codex work.
- `.gitignore`: ignores local OS, environment, build, coverage, and dependency artifacts.
- `README.md`: project overview, commands, assumptions, deployment, replay payload, and PR workflow.
- `index.html`: minimal app shell with canvas mount and compact restart/share controls.
- `src/main.ts`: bootstraps the game and exposes a small debug/test API.
- `src/styles.css`: fullscreen mobile-first canvas shell and compact icon controls.
- `src/game/EventHorizonGame.ts`: PixiJS scene, rendering, HUD, scaling, poster export, score submit call.
- `src/game/Simulation.ts`: deterministic gameplay state, phase changes, orbs, flybys, shadow arms, replay events.

- ``src/game/FixedStepLoop.ts``: fixed 60 Hz simulation loop decoupled from rendering.
- ``src/game/InputHandler.ts``: pointer/touch tap and swipe mapping to logical world coordinates.
- ``src/game/rng.ts``: string hash and ``mulberry32`` seeded RNG.
- ``src/game/posterizer.ts``: creates vertical share image from three gameplay frames.
- ``src/game/scoreClient.ts``: posts replay payloads to the score endpoint.
- ``src/game/constants.ts``, ``math.ts``, ``types.ts``, ``vite-env.d.ts``: shared constants, helpers, and types.
- ``netlify/functions/score-submit.mjs``: minimal score validation endpoint.
- ``gas/score-submit.gs``: Google Apps Script ``doPost`` JSON endpoint sample.
- ``netlify.toml``: Netlify build, publish, and functions directory config.
- ``package.json``, ``package-lock.json``: dependencies and runnable scripts.
- ``tsconfig.json``, ``vite.config.ts``, ``eslint.config.js``, ``playwright.config.ts``: TypeScript, Vite, lint, and browser test config.
- ``tests/**/*.ts``: deterministic replay, score endpoint, and mobile Chrome smoke tests.
- ``scripts/capture-artifacts.mjs``: captures mobile gameplay, poster, and collapse images.
- ``scripts/test-score-submit.mjs``: standalone endpoint sanity test.
- ``scripts/generate-report-pdf.mjs``: generates the requested PDF artifact.
- ``docs/artifacts/*``: compressed screenshots and poster image from the actual build.
- ``docs/pr-body.md``: PR description source.

Major Code Snippets

``index.html``

```
<main id="app" aria-label="Event Horizon game">
  <div id="game-shell">
    <div id="game-root" aria-label="Playable canvas"></div>
    <div id="status-layer" aria-live="polite">
      <button id="restart-button" type="button" aria-label="Restart run" title="Restart run">!></button>
      <button id="share-button" type="button" aria-label="Create share poster" title="Create share poster">!&#x2013;</button>
      <a id="poster-link" download="event-horizon-poster.png" aria-label="Download share poster">!&#x2013;</a>
    </div>
  </div>
</main>
<script type="module" src="/src/main.ts"></script>
```

Main Entry

```
const game = new EventHorizonGame(root, {
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000
});
```

```
await game.start();
```

Fixed-Step Loop

```
while (this.accumulatorMs >= FIXED_STEP_MS) {
  this.step(FIXED_STEP_MS);
  this.accumulatorMs -= FIXED_STEP_MS;
}

this.render(this.accumulatorMs / FIXED_STEP_MS);
```

``mulberry32``

```
export function mulberry32(seed: number): RandomSource {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let next = state;
    next = Math.imul(next ^ (next >>> 15), next | 1);
    next ^= next + Math.imul(next ^ (next >>> 7), next | 61);
    return ((next ^ (next >>> 14)) >>> 0) / 4294967296;
  };
}
```

PixiJS Init

```
await this.app.init({
  autoDensity: true,
  autoStart: false,
  backgroundAlpha: 0,
  preference: 'webgl',
```

```

    preserveDrawingBuffer: true,
    powerPreference: 'high-performance',
    resizeTo: this.root,
    resolution: Math.min(window.devicePixelRatio || 1, 2)
  });

```

Input Handler

```

if (moved >= this.swipeThresholdSq) {
  this.callbacks.onSwipe(this.start, end);
} else {
  this.callbacks.onTap(end);
}

```

Posterizer

```

const loadedFrames = await Promise.all(frames.slice(-3).map((frame) => loadImage(frame.dataUrl)));
context.fillText(`survived ${formatTime(stats.survivalMs)} • phase ${stats.phase}`, 52, footerY + 154);
return canvas.toDataURL('image/png', 0.86);

```

GAS `doPost`

```

function doPost(e) {
  var payload = JSON.parse(e.postData.contents || '{}');
  return ContentService.createTextOutput(JSON.stringify({
    ok: true,
    acceptedAt: new Date().toISOString(),
    score: Number(payload.score || 0),
    survivalMs: Number(payload.survivalMs || 0)
  })).setMimeType(ContentService.MimeType.JSON);
}

```

Sample Replay Payload

```

{
  "version": 1,
  "seed": "eh-2026-05-29-alpha",
  "startedAt": 1780051200000,
  "survivalMs": 67421,
  "score": 1280,
  "energyCaptured": 87,
  "maxStreak": 24,
  "tapEvents": [],
  "swipeEvents": [],
  "phaseTransitions": []
}

```

Test Results

- `npm run build`: passed. TypeScript compiled and Vite produced `dist`.
- `npm run lint`: passed.
- `npm run test`: passed, 2 files and 5 tests.
- `npm run test:e2e`: passed, 4 mobile Chrome tests.
- `npm run score:test`: passed, score endpoint returned status `200` and `{ "ok": true }`.
- Local Chrome smoke test: passed via Playwright and in-app browser visual inspection.
- Deterministic replay sanity: passed. Same seed plus same input timings produced identical replay payloads.
- Touch simulation sanity: passed. Tap and swipe events are captured in replay payloads.
- Posterizer export: passed. Browser test returns a `data:image/png;base64,...` poster.
- Score submit test: passed against `/.netlify/functions/score-submit` handler import.

Performance Notes

- The simulation uses a fixed 60 Hz step and avoids allocating per-frame input or entity arrays.
- Orb and flyby sprites use generated textures rather than drawing new art every frame.
- Background art is drawn once; per-frame `Graphics` clearing is limited to tethers, arms, HUD fill, and flyby trails.
- No heavy filters or physics engine are used.
- `preserveDrawingBuffer` supports poster screenshots but can cost some GPU performance; if poster export changes later, replacing it with targeted render-texture capture may be better.
- The main visible bottleneck risk is `canvas.toDataURL()` during poster capture; it is user-triggered rather than hot-loop work.

Deployment Notes

Netlify

- Build command: ``VITE_BASE_PATH=/ npm run build``
- Publish directory: ``dist``
- Functions directory: ``netlify/functions``
- Score function route: ``/.netlify/functions/score-submit``
- Branch deploys: connect the GitHub repo in Netlify and enable branch deploys for PR previews.
- Local Netlify dev: ``npx netlify dev`` from the repo root.

GitHub Pages

- Default Vite ``base`` is ``/event-horizon/``.
- Static build command: ``npm run build``
- Publish the generated ``dist/`` content through the chosen GitHub Pages workflow.
- Environment assumption: GitHub Pages uses a subpath, while Netlify uses root. Override with ``VITE_BASE_PATH=/`` for root deploys.

PR Summary

This PR introduces the first playable Event Horizon slice: deterministic simulation, PixiJS rendering, one-thumb input, replay payloads, phase-based collapse pressure, score submission samples, tests, documentation, screenshots, and this PDF report.

Screenshots and Poster

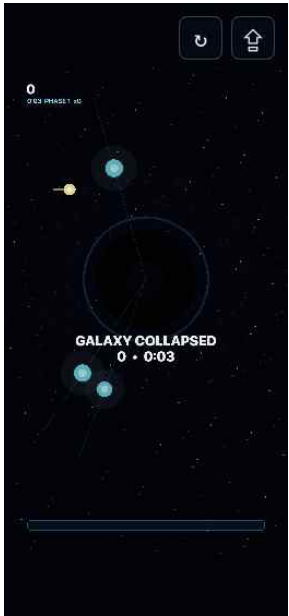
Gameplay mobile screenshot



Three-frame share poster



Collapse mobile screenshot



File Diffs

Diff Stat

```
docs/artifacts/collapse-mobile.jpg | Bin 31259 -> 5805 bytes
docs/artifacts/gameplay-mobile.jpg | Bin 24555 -> 4725 bytes
docs/artifacts/share-poster.jpg     | Bin 0 -> 6992 bytes
docs/artifacts/share-poster.png    | Bin 130509 -> 0 bytes
docs/first-pr-report.md             | 4 ++-
eslint.config.js                    | 4 +++-
scripts/capture-artifacts.mjs       | 29 ++++++-----
scripts/generate-report-pdf.mjs     | 7 +++-
8 files changed, 31 insertions(+), 13 deletions(-)
```

Unified Diff

```
diff --git a/docs/artifacts/share-poster.png b/docs/artifacts/share-poster.png
deleted file mode 100644
index 6ecf362..0000000
Binary files a/docs/artifacts/share-poster.png and /dev/null differ
diff --git a/docs/first-pr-report.md b/docs/first-pr-report.md
index 2f7e42c..b9f4e5b 100644
--- a/docs/first-pr-report.md
+++ b/docs/first-pr-report.md
@@ -72,7 +72,7 @@ AGENTS.md
 README.md
 docs/artifacts/collapse-mobile.jpg
 docs/artifacts/gameplay-mobile.jpg
-docs/artifacts/share-poster.png
+docs/artifacts/share-poster.jpg
 docs/first-pr-report.md
 docs/pr-body.md
 eslint.config.js
@@ -132,7 +132,7 @@ vite.config.ts
- `scripts/capture-artifacts.mjs`: captures mobile gameplay, poster, and collapse images.
- `scripts/test-score-submit.mjs`: standalone endpoint sanity test.
- `scripts/generate-report-pdf.mjs`: generates the requested PDF artifact.
-- `docs/artifacts/*`: screenshots and poster image from the actual build.
+- `docs/artifacts/*`: compressed screenshots and poster image from the actual build.
- `docs/pr-body.md`: PR description source.

## Major Code Snippets
diff --git a/eslint.config.js b/eslint.config.js
index 221d74e..b46adb3 100644
--- a/eslint.config.js
+++ b/eslint.config.js
@@ -30,7 +30,9 @@ export default tseslint.config(
   Request: 'readonly',
   Response: 'readonly',
   URL: 'readonly',
-   window: 'readonly'
+   window: 'readonly',
+   document: 'readonly',
+   Image: 'readonly'
 }
}

diff --git a/scripts/capture-artifacts.mjs b/scripts/capture-artifacts.mjs
index 821a5a0..298493b 100644
--- a/scripts/capture-artifacts.mjs
+++ b/scripts/capture-artifacts.mjs
@@ -6,8 +6,8 @@ await mkdir(outputDir, { recursive: true });

const browser = await chromium.launch({ channel: 'chrome' });
const page = await browser.newPage({
-   viewport: { width: 390, height: 844 },
-   deviceScaleFactor: 2,
+   viewport: { width: 260, height: 563 },
+   deviceScaleFactor: 1,
   isMobile: true,
   hasTouch: true
});
@@ -18,7 +18,7 @@ await page.waitForTimeout(1300);
await page.screenshot({
  path: new URL('gameplay-mobile.jpg', outputDir).pathname,
  type: 'jpeg',
-   quality: 58,
+   quality: 38,
  fullPage: false
});

@@ -38,18 +38,35 @@ const postDataUrl = await page.evaluate(() => window.__EVENT_HORIZON__?.export
  if (!postDataUrl || !postDataUrl.startsWith('data:image/png;base64, ')) {
    throw new Error('Poster export failed. ');
  }
}
- await writeFile(new URL('share-poster.png', outputDir), Buffer.from(postDataUrl.split(',')[1], 'base64'));
+ const posterJpeg = await page.evaluate(async (dataUrl) => {
+   const image = await new Promise((resolve, reject) => {
+     const candidate = new Image();
+     candidate.onload = () => resolve(candidate);

```



```

+   candidate.onerror = () => reject(new Error('Unable to load poster image.'));
+   candidate.src = dataUrl;
+ });
+ const canvas = document.createElement('canvas');
+ canvas.width = 270;
+ canvas.height = 480;
+ const context = canvas.getContext('2d');
+ if (!context) {
+   throw new Error('Unable to create poster compression canvas.');
```

```

+ }
+ context.drawImage(image, 0, 0, canvas.width, canvas.height);
+ return canvas.toDataURL('image/jpeg', 0.42);
+}, postDataUrl);
+await writeFile(new URL('share-poster.jpg', outputDir), Buffer.from(posterJpeg.split(',')[1], 'base64'));

await page.evaluate(() => window.__EVENT_HORIZON__?.forceEnd());
await page.waitForTimeout(950);
await page.screenshot({
  path: new URL('collapse-mobile.jpg', outputDir).pathname,
  type: 'jpeg',
- quality: 56,
+ quality: 36,
  fullPage: false
});

await browser.close();
console.log('Captured docs/artifacts/gameplay-mobile.jpg');
-console.log('Captured docs/artifacts/share-poster.png');
+console.log('Captured docs/artifacts/share-poster.jpg');
console.log('Captured docs/artifacts/collapse-mobile.jpg');
diff --git a/scripts/generate-report-pdf.mjs b/scripts/generate-report-pdf.mjs
index 5cab74b..1b5ce04 100644
--- a/scripts/generate-report-pdf.mjs
+++ b/scripts/generate-report-pdf.mjs
@@ -9,7 +9,7 @@ const reportPath = new URL('first-pr-report.md', docsDir);
const pdfPath = new URL('first-pr-report.pdf', docsDir);
const artifacts = [
  new URL('artifacts/gameplay-mobile.jpg', docsDir),
- new URL('artifacts/share-poster.png', docsDir),
+ new URL('artifacts/share-poster.jpg', docsDir),
  new URL('artifacts/collapse-mobile.jpg', docsDir)
];

@@ -24,10 +24,9 @@ const diffTextRaw = runGit([
  '.',
  ':(exclude)package-lock.json',
  ':(exclude)docs/artifacts/*.jpg',
- ':(exclude)docs/artifacts/*.png',
  ':(exclude)docs/first-pr-report.pdf'
]);
-const maxDiffChars = 70000;
+const maxDiffChars = 8000;
const diffText =
  diffTextRaw.length > maxDiffChars
    ? `${diffTextRaw.slice(0, maxDiffChars)}\n\n[Diff truncated for PDF practicality. package-lock and binary images are
represented in the stat above.]\n`
@@ -133,7 +132,7 @@ function addScreenshots(document) {
  document.font('Helvetica-Bold').fontSize(10).text(labels[index]);
  document.moveDown(0.2);
  document.image(image.pathname, {
-   fit: index === 1 ? [220, 300] : [180, 300],
+   fit: index === 1 ? [160, 230] : [130, 230],
    align: 'center'
  });
  document.moveDown(0.7);

```